

E5 Hybrid Search, Explained Simply

How `e5_hybrid.py` finds the right section of SOP 214 by combining a keyword judge (the register) with a meaning judge (the toy foreman). All numbers below are real outputs on the 10 heading chunks from E4.

1. The job

Someone types a question. The code must score each of the 10 SOP sections for how well it matches, then return the top 3. Two judges score independently, their scores are put on the same scale, blended 60/40, and the best three win.

2. Setup: cleaning words

Every piece of text first goes through **words()**: lowercase it, split into words (keeping hyphens so 'e-way' stays whole), drop filler like 'the' and 'what' (the **STOP** list), and fold word forms together ('re-filed' becomes 're-filing', the **STEM** table). A small **SYN** table maps everyday words to SOP words: parcel to consignment, stuck to hold, paperwork to document, problem to mismatch. Only the foreman uses SYN.

Input	After words()
parcel stuck, paperwork problem, what should I do	parcel, stuck, paperwork, problem
The e-way bill was re-filed	e-way, bill, was, re-filing

3. The register: keyword scoring (bm25ish)

Naive idea: count how often the question's words appear in each section. It fails. For 'e-way bill re-filing', section 3 says 'bill' 12 times and section 4 only 9 times, so section 3 would win even though the question is about re-filing. BM25 is this count with two fixes.

Fix A, rare words count more. For each word, ask how many of the 10 sections contain it (df). Few sections means rare means a heavy weight. Like searching a crowd: 'wears shoes' is useless, 'wears a red turban' is gold.

Fix B, repeats stop counting. Squash the raw count with $\text{count} / (\text{count} + 1.2)$. The first mention is worth a lot, the ninth almost nothing, and it can never pass 1.0. Like glasses of water when thirsty.

Word	In how many sections	Rarity weight	Raw count	Squashed
e-way	7 of 10	0.38	1	0.45
bill	7 of 10	0.38	3	0.71
re-filing	5 of 10	0.69	9	0.88
(a word in all 10)	10 of 10	0.05	12	0.91

Score for a section = sum over question words of (squashed count x rarity weight). Section 4 versus section 3:

Section 4: Re-filing	count	squashed	weight	adds	Section 3: Cancellation	count	squashed	weight	adds
e-way	4	0.77	0.38	0.29	e-way	4	0.77	0.38	0.29
bill	9	0.88	0.38	0.34	bill	12	0.91	0.38	0.35
re-filing	4	0.77	0.69	0.53	re-filing	1	0.45	0.69	0.31
Total				1.17	Total				0.96

Section 4 now wins. The 'bill' rows are nearly equal thanks to Fix B. The 're-filing' row decides it thanks to Fix A. **Blind spot:** the register matches spelling only. It cannot know that 'parcel' means 'consignment'.

Remember: BM25 is word counting with two adjustments. Rare words count more. Repeats stop mattering after a few.

4. The toy foreman: matching by meaning (toy_embed, cosine, semantic)

Real systems send text to a paid model that returns a long list of numbers; similar meanings get similar lists. This file fakes that for free with the SYN table. **toy_embed()** cleans the text, **throws away any word with a hyphen or digit** ('e-way', 're-filing', 'kf-2481' vanish, imitating how real embedders stumble on codes), swaps in synonyms, and counts what is left into a bag.

Question	Bag the foreman sees
parcel stuck, paperwork problem, what should I do	consignment 1, hold 1, document 1, mismatch 1 (all four translated)
e-way bill re-filing KF-2481	bill 1 (three of four words dropped: blind to codes)

cosine() compares two bags and returns 0 to 1. Think of bags as recipes: do they use the same mix of ingredients, ignoring batch size? Top = multiply matching counts and add. Bottom = size of each bag, multiplied. Dividing by the bottom is what stops long sections winning just for being long. Worked example:

	consignment	hold	document	mismatch	officer
question (a)	1	1	1	1	0
section (b)	3	2	0	0	4
multiplied	3	2	0	0	0

Top = 3 + 2 = 5. Size of a = $\sqrt{1+1+1+1}$ = 2.0. Size of b = $\sqrt{9+4+16}$ = 5.39. Cosine = 5 / (2.0 x 5.39) = **0.46**. About half alike: two shared ideas, plus a lot of 'officer' the question never mentioned.

semantic() embeds the question once, embeds each section, returns 10 cosines. It first checks the **DRY_RUN** environment variable and stops the program if it is unset, so nobody hits the paid path by accident. Real result for 'parcel stuck, paperwork problem':

Cosine	Section	consignment	hold	document	mismatch
0.584	1. What a customs hold is	4	9	2	4
0.316	Preamble	3	1	2	0
0.314	8. Message templates	3	2	1	0
0.207	4. Re-filing	3	5	0	0
0.000	5. Vehicle updates	0	0	0	0

Section 1 wins clearly: it is the only section holding all four translated ideas, and it is the right place to send someone saying 'my parcel is stuck'. The register found section 1 only via the literal word 'problem'.

5. Fusing the two (norm, hybrid) and running

The judges use different scales: the register's best score is 1.17, the foreman's is 0.58. Adding them raw would let the register dominate. **norm()** divides every score by the list's maximum, so each judge's favourite becomes exactly 1.0 (the 'or 1' avoids dividing by zero when all scores are zero). **hybrid()** then blends:

```
fused = 0.6 x register_norm + 0.4 x foreman_norm      # w_sem = 0.4: foreman gets 40% of the say
```

Like two interviewers scoring out of 10, with the senior one trusted 60/40. Then sort sections by fused score, keep the top k=3, and return each winner's label line. Real fused tables:

'e-way bill re-filing'	Reg	For	Fused	'parcel stuck, paperwork problem'	Reg	For	Fused
4. Re-filing	1.00	0.80	0.92	1. What a customs hold is	1.00	1.00	1.00
3. Cancellation	0.82	1.00	0.89	Preamble	0.27	0.54	0.38
2. First checks	0.69	0.59	0.65	8. Message templates	0.27	0.54	0.38
5. Vehicle updates	0.63	0.32	0.50	9. Audit	0.00	0.39	0.16

Left: the foreman actually preferred section 3 (it only saw 'bill'), but the register saw 're-filing' four times and, with 60% of the vote, pulled section 4 to the top. Right: both agree on section 1. Sections 9 and 3 scored zero with the register yet keep a small fused score because the foreman found meaning there. Each judge covers the other's blind spot.

The main block runs two queries, one jargon-heavy and one in everyday words. Actual output:

```
$ DRY_RUN=1 python e5_hybrid.py
q: e-way bill re-filing
['[SOP 214 - 4. Re-filing an e-way bill after a hold]', '[SOP 214 - 3. Cancellation of an e-way bill]', '[SOP 214 - 2. First checks before any action]']
q: parcel stuck, paperwork problem, what should I do
['[SOP 214 - 1. What a customs hold is]', '[SOP 214 - Preamble]', '[SOP 214 - 8. Message templates]']
$ python e5_hybrid.py      # without the flag
real_embed is the paid path: set DRY_RUN=1 for the toy foreman
```

Remember: hybrid search is two judges, scaled to the same range, blended with a weight, top few kept. The register catches jargon and codes. The foreman catches everyday words. Neither alone gets both queries right. Together they do.